Compare Contiguous and Linked File Allocation methods in terms of access speed, memory utilization, and fragmentation. Identify which method is more suitable for sequential file access and justify your answer.

Ans:

=) Contiguous allocation stores all blocks of a file in consecutive disk locations, whereas linked allocation stores the file blocks anywhere on the disk and links them using pointer.

Feature	Contiguous Allocation.
Storage	Blocks are stored continuously.
Access speed	Very fast, especially sequential and direct access.
Memory utilization	May waste space due to external fragmentation.
Fragmentation	Suffers from external fragmentation.
Overhead	Very little overhead.

Feature	Linked Allocation
Storage	Blocks can be stored anywhere
Access speed	Slower because pointers must be [illegible]
Memory utilization	Uses available blocks efficiently
[illegible]	[illegible]

Contiguous Allocation

→ In contiguous allocation, if a file requires 5 blocks, the blocks may be stored at: 10 → 11 → 12 → 13 → 14.

* Since the blocks are adjacent, the disk head can access them quickly.

Linked Allocation:-

→ In linked allocation, the blocks may be:

10 → 25 → 7 → 40 → 18.

* Each block contains a pointer to the next block. Therefore, accessing a particular block requires following the chain.

⇒ Contiguous allocation is more suitable for sequential file access.

Reason:

1. Blocks are stored next to each other.
2. Disk head movement is reduced.
3. Sequential reading is faster.

---

2) Differentiate between Linked and Indexed File Allocation with respect to random access capability, storage overhead and reliability. Explain which method is more efficient for large files.

Ans * Linked allocation stores pointers with individual file blocks, while indexed allocation uses a separate index block containing the addresses of all blocks belonging to a file.

Feature	Linked Allocation	Indexed Allocation
Random access	Poor	Excellent
Storage overhead	Pointer in every block	Index block required
Block Organization	Blocks can be anywhere	Blocks can be anywhere.
Reliability	Pointer failure can break the chain.	Central index can be a single point of a failure.
Large files	Less efficient for random access	More efficient.
Accessing a block	Follow pointers sequentially	Directly obtain address from index

Linked Allocation:

Example:

Block 5 → Block 19 → Block 6 → Block 25 → Block 40.

* To reach Block 25, the system has to follow the chain ~~me~~ from the beginning.

Therefore, random access is slow.

Index Block:

entry	Block Address
1	5
2	19
3	6
4	25
5	40

* The Operating System can directly locate the required block using the index.

Storage Overhead:

* Linked allocation:- Every blocks needs a pointer to the next block.

* Indexed allocation:- Requires an additional index block or multiple index blocks for a very large files.

Reliability:

⇒ Linked allocation depends on the pointer chain. If a pointer is corrupted, the remaining part of the file may become inaccessible.

⇒ ∴ Indexed allocation is generally more efficient for large files, particularly when random or direct access is required.

Compare FCFS Vs SSTF Disk Scheduling.

Ans

* FCFS (First come, First served) processes requests in their arrival order. SSTF (Shortest Seek Time First) selects the request closest to the current head position.

Feature	FCFS	SSTF
Seek time	High	Lower.
Fairness	Very high	Lower.
Starvation	No	Possible
Implementation	Simple	More complex
Heavy load	Less efficient	More efficient

* Requests are serviced in the order they arrive. It is fair but may cause large head movements.

SSTF:

* The closest request is serviced first, reducing average seek time and improving performance.

Under heavy load:-

⇒ SSTF generally performs better because it reduces disk head movement and average seek time.

- However, SSTF may cause starvation for requests located far from the current head.

* FCFS is fairer, while SSTF provides better performance and lower seek time.

SCAN Vs C-SCAN.

⇒ SCAN moves the disk head in one direction, services requests, and then reverse direction.

⇒ C-SCAN services requests in only one direction and returns to the beginning before continuing.

Scan:

* It works like an elevator. The head moves towards one end and services requests, then reverse.

C-SCAN:

* The head moves in one-direction. After reaching the end, it returns to the begining and continues servicing.

Feature	SCAN	C-SCAN.
Head movement	Both directions	One-Servicing direction.
Response time	Good	More uniform.
Fairness	Good	Very good.
Waiting time	Less predictable	More predictable.
Service.	Bidirectional	Circular.

=) C-SCAN is preferable when predictable and uniform response time is important.

LOOK vs C-LOOK.

=) LOOK and C-LOOK are improved versions of SCAN and C-SCAN. They do not travel to the physical end of the disk if there are no requests there.

LOOK:

* The head moves in one direction until the last pending request, then reverse.

C-LOOK: The head moves in one direction until the last request, then jumps to the first request and continues the same direction.

Feature	LOOK	C-LOOK
Direction	Both directions	One-direction
Head movement	Low	Low
Efficiency	High	High
Waiting time	Good	More uniform
Implementation	Simple	Slightly complex

Performance:

- LOOK → better when reducing total head movement is important.
- C-LOOK → better when uniform waiting time is important